BCA Semester 1 Examination 2025

Model Solutions: 1BCA3 - Programming Logic & Design Techniques

Session	Paper Code	Time	Full Marks	Pass Marks
2025-2028	111603	3 Hours	100	40

Instructions: Answer any five questions (equal marks). Solutions for all ten questions are given below for practice. Write answers in your own words in the exam.

Q1. Define the following terms: (i) Time Complexity (ii) Algorithm (iii) Variable and Constant (iv) Control Structure

(i) Time Complexity

Time complexity is the amount of time an algorithm takes to run as a function of the size of its input (n). It is not measured in seconds but by counting the number of basic operations, and is expressed with asymptotic notation such as O(1), O(log n), O(n), O(n log n), O(n^2).

(ii) Algorithm

An algorithm is a finite, step-by-step, well-defined sequence of instructions used to solve a problem or perform a task. **Characteristics:** input, output, definiteness (each step is clear), finiteness (it ends), effectiveness (each step is doable).

(iii) Variable and Constant

Variable	Constant
A named memory location whose value can change during execution.	A fixed value that cannot change during execution.
Example: int age = 18; later age = 19;	Example: const float PI = 3.14; or the literal 100.

(iv) Control Structure

Control structures decide the order in which statements of a program are executed. The three basic types are:

- **Sequence:** statements run one after another.
- **Selection (decision):** a path is chosen on a condition - if, if-else, switch.
- **Iteration (loop):** statements repeat while a condition holds - for, while, do-while.

Q2. Define flow chart. Explain symbols used in a flow chart and describe advantages of flow chart with example.

Definition

A flowchart is a diagrammatic (pictorial) representation of an algorithm or process using standard symbols connected by arrows to show the sequence of steps and the flow of control.

Flowchart symbols

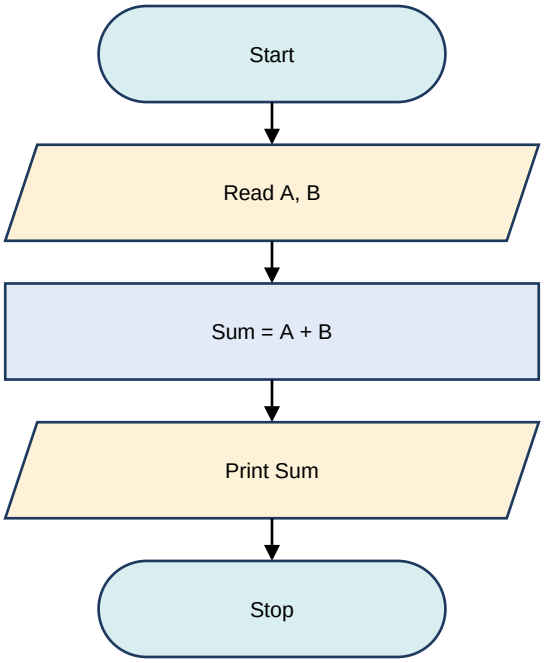
Symbol	Name	Purpose
Oval / rounded rectangle	Terminator	Start and End of the flowchart
Parallelogram	Input / Output	Reading data and displaying results

Symbol	Name	Purpose
Rectangle	Process	Calculations and assignments
Diamond	Decision	Condition with Yes/No (True/False) outcomes
Arrow (flow line)	Flow line	Shows direction of flow
Small circle	Connector	Joins parts of a flowchart (same or other page)
Double-sided rectangle	Predefined process	Calls a subroutine/function
Hexagon	Preparation	Loop setup (initialisation)

Advantages

- Easy to understand the logic of a program at a glance; better communication between programmers and users.
- Helps in planning, analysis and finding logical errors before coding (debugging).
- Acts as documentation and makes maintenance and modification easy.
- Language independent and can be converted directly into a program.

Example: flowchart to find the sum of two numbers



Q3. Write an algorithm and draw a flow chart to check whether the unit-place digit of a given number is even or odd.

Algorithm

1 Start.

2 Read the number N.

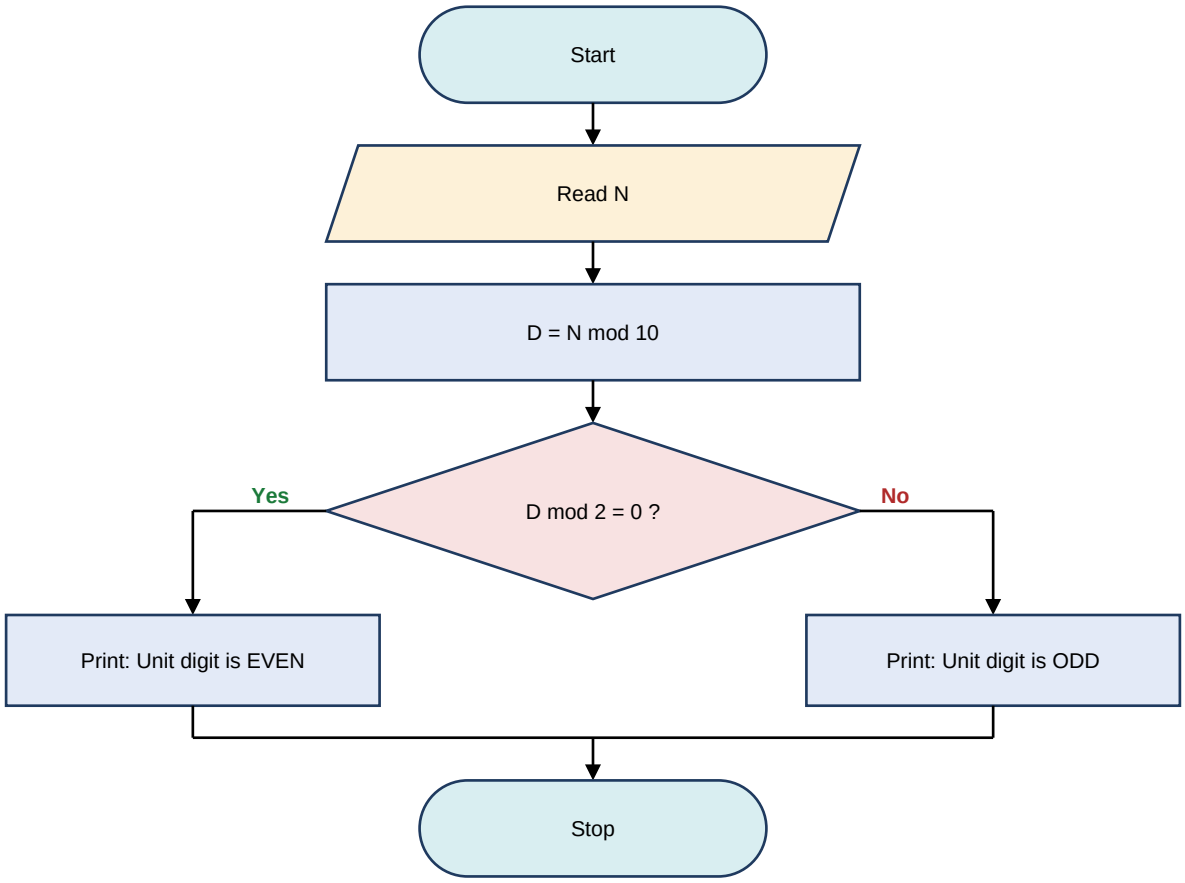
3 Find the unit-place digit: D = N mod 10 (use the absolute value if N may be negative).

4 If D mod 2 = 0 then print 'Unit digit is EVEN', otherwise print 'Unit digit is ODD'.

5 Stop.

Example: N = 3847, D = 3847 mod 10 = 7, 7 mod 2 = 1, so the unit digit is ODD.

Flowchart



Q4. Discuss different problem solving techniques.

Problem solving is the process of understanding a problem and designing steps to solve it. General steps: **define the problem, analyse it, develop an algorithm/flowchart, code, test and debug, document and maintain.**

Common techniques

Technique	Idea	Example
Algorithm / Pseudocode / Flowchart	Describe the solution step by step before coding.	Algorithm to find the largest of three numbers
Divide and conquer / Decomposition	Break a big problem into smaller sub-problems, solve each, combine results.	Merge sort, binary search
Brute force	Try all possible solutions and pick the correct one.	Linear search, password guessing
Greedy method	Choose the best option at each step hoping for a global optimum.	Coin change, Dijkstra, Huffman coding
Dynamic programming	Store results of overlapping sub-problems to avoid recomputation.	Fibonacci, knapsack
Backtracking	Build the solution step by step and undo (backtrack) when a step fails.	N-Queens, maze solving

Technique	Idea	Example
Recursion	A problem is solved using smaller instances of itself.	Factorial, Tower of Hanoi
Top-down and bottom-up	Start from the main problem and refine, or build from small modules upward.	Modular programming
Heuristic / trial and error	Rule-of-thumb or experimental approach when exact methods are costly.	Game playing, route finding

Q5. Explain procedural and recursive programming.

Procedural programming

In procedural programming a program is divided into a set of procedures (functions/subroutines) that are executed in sequence using variables, selection and loops. The focus is on the steps (*how* to do it). Data and functions are separate. Languages: C, Pascal, FORTRAN, BASIC.

- Top-down design, code reusability through functions, easy to trace.
- Repetition is done mostly with loops (iteration).

Recursive programming

Recursion is a technique in which a function calls itself to solve a smaller version of the same problem. Every recursive function needs (1) a **base case** that stops the recursion and (2) a **recursive case** that moves towards the base case.

```
int fact(int n)
{
    if (n == 0 || n == 1)     /* base case      */
        return 1;
    return n * fact(n - 1);   /* recursive case */
}
/* fact(4) = 4 * fact(3) = 4*3*fact(2) = 4*3*2*fact(1) = 24 */
```

Basis	Procedural (iterative)	Recursive
Repetition	Loops (for, while)	Function calls itself
Termination	Loop condition becomes false	Base case reached
Memory	Less (no stack growth)	More (stack frame for each call)
Code	Longer but faster	Short and clear for tree/divide problems; slower

Q6. Explain complexity of an algorithm.

Complexity measures the resources an algorithm needs as the input size n grows. It helps to compare algorithms and choose the most efficient one.

- **Time complexity:** number of operations/steps executed as a function of n.
- **Space complexity:** amount of extra memory required (variables, arrays, recursion stack) as a function of n.

Cases

- **Best case:** minimum time (e.g. element found at first position in linear search).

- **Average case:** expected time over all inputs.
- **Worst case:** maximum time (element absent or last); normally used for analysis.

Notation	Name	Example
O(1)	Constant	Accessing an array element
O(log n)	Logarithmic	Binary search
O(n)	Linear	Linear search, finding maximum
O(n log n)	Linearithmic	Merge sort, quick sort (average)
O(n^2)	Quadratic	Bubble sort, nested loops
O(2^n)	Exponential	Recursive Fibonacci, subset generation

Example: a single loop `for(i=0;i<n;i++) sum+=a[i];` executes n times, so the time complexity is O(n) and the space complexity is O(1).

Q7. What do you mean by Asymptotic? Explain all.

Asymptotic analysis describes the behaviour of an algorithm's running time when the input size n becomes very large, ignoring constants and lower-order terms. Asymptotic notations are mathematical tools to express this growth rate.

1. Big-O notation, O(g(n)) - upper bound (worst case)

f(n) = O(g(n)) if there exist positive constants c and n0 such that f(n) <= c*g(n) for all n >= n0. Example: 3n + 2 = O(n) because 3n + 2 <= 4n for n >= 2.

2. Omega notation, Omega(g(n)) - lower bound (best case)

f(n) = Omega(g(n)) if f(n) >= c*g(n) for all n >= n0. Example: 3n + 2 = Omega(n).

3. Theta notation, Theta(g(n)) - tight bound (average/exact growth)

f(n) = Theta(g(n)) if c1*g(n) <= f(n) <= c2*g(n) for all n >= n0, i.e. f is both O(g) and Omega(g). Example: 3n + 2 = Theta(n).

4. Little-o and little-omega

Little-o, o(g(n)): f grows strictly slower than g (n = o(n^2)). Little-omega, w(g(n)): f grows strictly faster than g (n^2 = w(n)).

Notation	Bound	Meaning
O	Upper	Algorithm takes at most this time
Omega	Lower	Algorithm takes at least this time
Theta	Tight	Algorithm takes exactly this order of time
o / w	Strict upper / lower	Strictly smaller / larger growth

Q8. Explain the purpose of Expression, Statement and Loop in a program.

Expression

An expression is a combination of constants, variables, operators and function calls that evaluates to a single value. Its purpose is to perform calculations and comparisons. Examples: `a + b * 2`, `x > 10 && y < 5`.

Statement

A statement is a complete instruction that tells the computer to perform an action; in C it ends with a semicolon. Types: declaration (`int a;`), assignment (`a = b + 2;`), input/output, control (if, switch), loop and function-call statements. A program is a sequence of statements; an expression is often a part of a statement.

Loop

A loop repeats a block of statements as long as a condition is true, so the same code need not be written again and again. Purpose: reduce code, process arrays/lists, and repeat tasks. Types: **for** (known count), **while** (entry-controlled) and **do-while** (exit-controlled).

```
for (i = 1; i <= 5; i++)      /* loop            */
    sum = sum + i;           /* statement       */
                             /* (sum + i) is an expression */
```

Q9. Write short notes on any two: (i) Encapsulation (ii) Fibonacci series (iii) Pseudo code

(i) Encapsulation

Encapsulation is an object-oriented concept in which data (attributes) and the functions (methods) that work on that data are bound together into a single unit called a class, and direct access to the data is restricted using access specifiers (private, protected, public). **Benefits:** data hiding and security, easier maintenance, modularity, controlled access through getter/setter methods. Example: a BankAccount class keeps *balance* private and allows changes only through deposit() and withdraw().

(ii) Fibonacci series

A series in which each term is the sum of the two preceding terms, starting with 0 and 1: **0, 1, 1, 2, 3, 5, 8, 13, 21, ...** (F(n) = F(n-1) + F(n-2), F(0) = 0, F(1) = 1).

1. Start; read n (number of terms).
2. Set a = 0, b = 1, count = 0.
3. Repeat while count < n: print a; next = a + b; a = b; b = next; count = count + 1.
4. Stop.

(iii) Pseudo code

Pseudocode is an informal, language-independent description of an algorithm written in plain English with programming-like keywords (IF, ELSE, WHILE, FOR, READ, PRINT). It cannot be compiled but is easy to read and later convert into code.

```
BEGIN
  READ a, b
  IF a > b THEN
      PRINT a, "is larger"
  ELSE
      PRINT b, "is larger"
  ENDIF
END
```

Q10. Write an algorithm and draw a flow chart to check whether a triangle is right angled or not.

Idea

A triangle with sides a, b, c is right angled if the square of one side equals the sum of the squares of the other two (Pythagoras theorem): a^2 + b^2 = c^2 (c being the longest side). The check is done for all three combinations so the sides can be entered in any order.

Algorithm

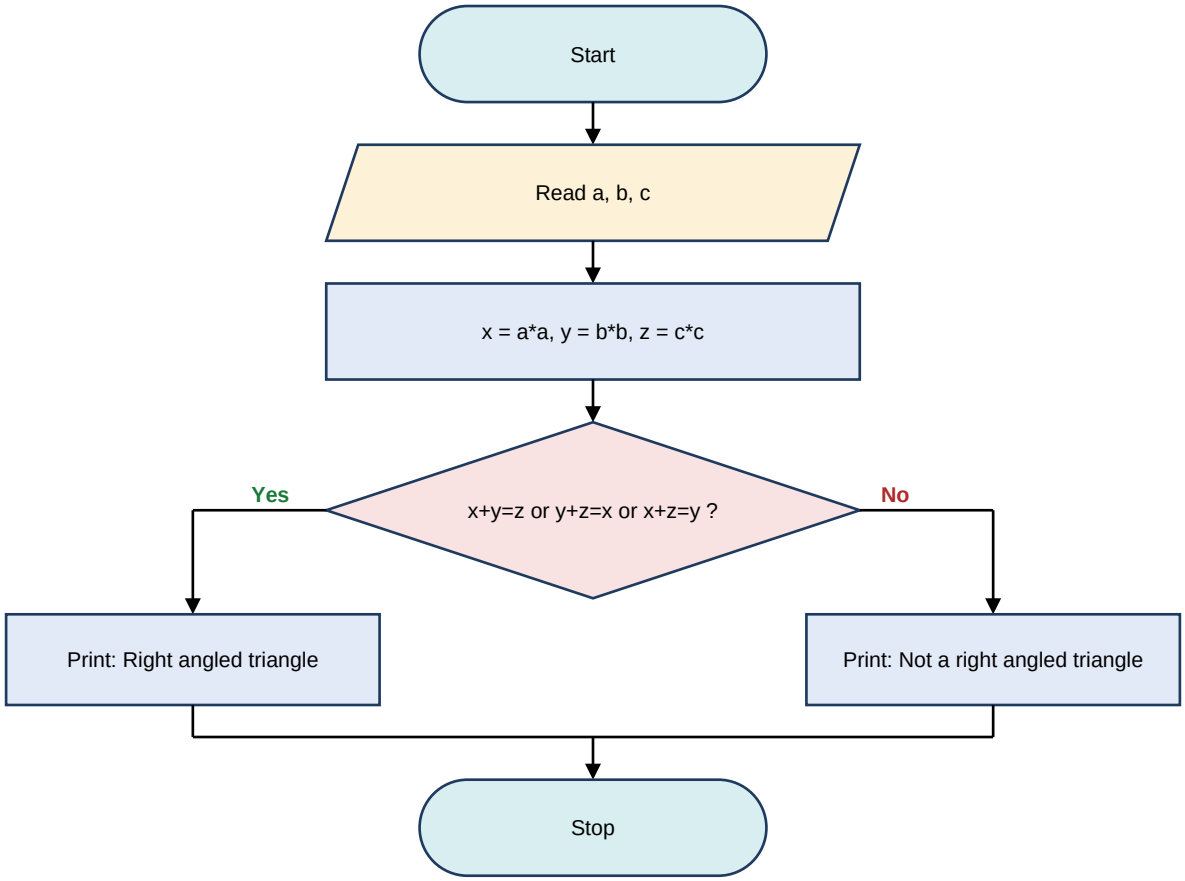
1 Start.

2 Read the three sides a, b, c.

3 Compute x = a*a, y = b*b, z = c*c.

4 If (x + y = z) or (y + z = x) or (x + z = y) then print 'Right angled triangle', otherwise print 'Not a right angled triangle'.

5 Stop.

Example: sides 3, 4, 5 give x = 9, y = 16, z = 25; x + y = 25 = z, so it is right angled. Sides 4, 5, 6 give 16 + 25 = 41 <> 36, so it is not.

Flowchart